

Final Exam for CSC 6301 SU 1 2026

You have spent the last seven weeks learning how to move from "writing code" to "designing software systems." Take your time to reflect on these principles—***you've got this!***

Instructions: Please answer any 10 of the following 14 questions. Each answer should be concise (one paragraph or a short list of points).

NOTE: Only your first 10 answers will be graded. There is NO extra credit for extra answers. You have a time limit of 4 hours to complete after you have started.

I would suggest that you spend your first 5 minutes going through the questions and decide on which 10 questions you want to answer. I would strongly recommend to time yourself and limit your time to a maximum of 19 minutes per question. Each of your answers is worth 10 points.

The exam is open book, open notes, but please do not access the general search within the Internet

NOTE:

This quiz contains 14 questions. You are only required to answer 10. Please choose 4 questions to leave entirely blank.

1. **Readability vs. Specification in Software Maintenance** : In Week 1, the course distinguished between documentation for "Readability" and documentation for "Specification". Compare these two documentation targets in terms of their primary audiences, structural objectives, and how they directly mitigate "software rot" during the maintenance and evolution phases of the Software Development Lifecycle (SDLC).

Answer

Documentation for **readability** and **specification** serve distinct but complementary roles in software engineering. Readability-oriented documentation is primarily intended for developers maintaining or modifying the code, focusing on internal logic, structure, and implementation details such as algorithms, invariants, and dependencies. Its objective is to facilitate maintenance tasks like bug fixing or feature enhancement. In contrast, specification-oriented documentation targets external users or developers who wish to reuse the module, emphasizing the public interface inputs, outputs, and behaviors without exposing internal implementation. Together, they mitigate software rot by ensuring that both the internal mechanics and external usage remain understandable over time, preventing degradation caused by poorly understood or undocumented systems during the maintenance and evolution phases of the SDLC. Additionally, readability focuses on the code as an "open box," while specification treats it as a "black box," reinforcing both maintenance and reuse perspectives.

2. **Object-Oriented Enforcement & Architectural Reusability** : Python allows an object-oriented approach, whereas Java strictly enforces it by requiring all code to exist within a class. Analyze how Java's reliance on static typing and mandatory class-based architecture serves as an engineering tool designed for large-scale enterprise software reuse, contrasting it with Python's rapid prototyping paradigms.

Answer

Java enforces object-oriented programming through mandatory class-based structures and static typing, which acts as a disciplined engineering framework for large-scale software reuse. By requiring explicit type declarations and encapsulating all behavior within classes, Java ensures consistency, predictability, and easier integration across large enterprise systems. This rigidity promotes strong architectural design and minimizes runtime ambiguity. In contrast, Python supports object-oriented programming but does not enforce it, favoring flexibility and rapid prototyping. While Python enables faster development and experimentation, Java's strict structure makes it more suitable for long-term, maintainable, and reusable systems where clear contracts and stability are critical for collaboration across teams.

This makes Java particularly suitable for large-scale systems where software reuse is critical, as it enforces object-oriented consistency across all components.

3. The Mechanics of Professional API Documentation (Javadocs) :

Explain the structural role of professional Javadoc documentation tags (@param, @return, @author, and @version) in software engineering. Specifically, explain how the javadoc compiler tool parses these elements to create a detached "owner's manual" for an external development team trying to integrate a pre-built class without viewing its underlying implementation.

Answer

Javadoc tags such as **@param**, **@return**, **@author**, and **@version** structure documentation in a standardized format that the javadoc compiler parses to generate a complete HTML-based documentation system. Each tag serves a precise purpose: @param documents input parameters, @return specifies output behavior, and metadata tags like @author and @version provide traceability. The compiler extracts these structured comments to create a detached, human-readable "owner's manual" that describes how to use a class or method without requiring access to its source code. This allows external developers to integrate and reuse software components efficiently while respecting encapsulation and abstraction principles. The documentation is written before class or method definitions and follows a strict format that the compiler recognizes and converts into structured HTML pages.

4. **Inheritance vs. Composition in Class Architectures** : A common pitfall in Object-Oriented design is using inheritance incorrectly just to reuse code without considering the semantic hierarchy. Define the conceptual difference between a strict "Is-A" relationship and a "Has-A" relationship. Provide a scenario where choosing composition over inheritance provides a more maintainable architecture when unexpected external feature modifications occur.

Answer

Inheritance represents an **"Is-A" relationship**, where a subclass is a specialized form of a superclass and inherits its behavior, while composition represents a **"Has-A" relationship**, where a class contains instances of other classes as components. A common misuse of inheritance occurs when it is applied only for code reuse without a true hierarchical relationship. For example, instead of making a "NotificationSystem" inherit from "EmailService," composition would allow the system to contain different notification strategies (Email, SMS, WhatsApp). Composition is more maintainable because it allows changes—such as adding a new notification method—without modifying the existing hierarchy, thereby supporting flexibility and reducing the impact of unexpected feature changes.

This approach follows the principle of favoring composition over inheritance, which improves flexibility and reduces tight coupling.

5. **Polymorphism and Interface-Driven Design** : Using the principles discussed in Week 3, explain how Java Interfaces facilitate polymorphism. How does designing a software system around small, cohesive interfaces (as opposed to concrete class implementations) decouple high-level application modules from lower-level utility extensions?

Answer

Java interfaces enable polymorphism by defining a contract that multiple classes can implement, allowing different objects to be treated uniformly based on shared behavior rather than concrete implementation. This approach decouples high-level modules from low-level implementations because the system depends on abstract interfaces instead of specific classes. By designing small, cohesive interfaces, developers can easily extend functionality by introducing new implementations without modifying existing code. This aligns with SOLID principles, particularly the Dependency Inversion Principle, ensuring that systems remain modular, extensible, and easier to maintain. This ensures that high-level modules depend on abstractions rather than concrete implementations, reducing dependency coupling.

6. **Structural Relationships: Composition vs. Aggregation** : Within the realm of structural software dependencies, distinguish between "Composition" and "Aggregation". Address their differences in terms of object lifecycles, memory ownership, and explain how a UML class diagram uniquely visualizes each relationship.

Answer

Composition and aggregation represent two forms of "has-a" relationships with different levels of ownership and lifecycle dependency. In **composition**, the contained object is fully owned by the parent class and cannot exist independently; if the parent is destroyed, the child is also destroyed. In **aggregation**, the contained object exists independently and can be shared across multiple parent objects. In UML diagrams, composition is represented by a **filled diamond**, indicating strong ownership, while aggregation uses a **hollow diamond**, indicating weaker association. These distinctions are crucial for understanding memory management, object lifecycle, and system design. Composition implies strong ownership and lifecycle dependency, while aggregation represents a weaker association with shared ownership.

7. **Software Reuse Typologies and the Vulnerabilities of Forked Code**

: The research community classifies software reuse into several kinds, including Opportunistic, Planned, Referenced, and Forked. Define "Forked" reuse and evaluate why it is heavily discouraged in professional software engineering environments compared to "Referenced" reuse, particularly regarding upstream patching and version alignment.

Answer

"Forked" reuse occurs when developers copy existing code into a new project and modify it independently rather than referencing the original source. While initially convenient, forked reuse is discouraged because it disconnects the code from upstream updates, bug fixes, and improvements. In contrast, "referenced" reuse maintains a dependency on the original source, allowing projects to benefit from continuous updates. Forked code leads to version fragmentation, duplicated maintenance effort, and increased risk of outdated or insecure software, making it inefficient and unsustainable in professional environments.

Additionally, forked reuse prevents benefiting from future patches, security updates, and version alignment with the original source.

8. **The Strategy of Refactoring vs. The Nuclear Rewrite** : During the Maintenance & Evolution phase of the SDLC, a team is faced with a major, unexpected client requirement change. Compare the software engineering strategies of "Refactoring" and "Rewriting". Why is refactoring highly preferred for preserving specialized domain knowledge, and under what exact conditions should a total rewrite be chosen?

Answer

Refactoring involves improving the internal structure of existing code without altering its external behavior, while rewriting involves discarding the existing system and building a new one from scratch. Refactoring is preferred because it preserves domain knowledge, existing functionality, and tested behavior, allowing incremental improvements with lower risk. It supports continuous maintenance and aligns with cost-effective development practices. A complete rewrite should only be considered when the system suffers from severe technical obsolescence, lacks any meaningful structure, or when requirements have fundamentally changed to the point where the existing architecture becomes more of a hindrance than a foundation.

Refactoring keeps the system in the maintenance phase, while rewriting effectively restarts the SDLC from the beginning.

9. **Structural and Behavioral Mechanics of the Observer Pattern :**

Describe the fundamental architectural goal of the Observer Design Pattern. Map out how it handles the structural dependencies between a "subject" and its "observers", and explain how this pattern enforces compliance with the Open-Closed Principle (OCP) within the SOLID guidelines.

Answer

The Observer pattern establishes a one-to-many relationship between a **subject** and multiple **observers**, where any change in the subject's state automatically triggers notifications to all registered observers. Structurally, the subject maintains a list of observers and provides methods to add, remove, and notify them. This design adheres to the Open-Closed Principle because new observer types can be added without modifying the subject's implementation. It decouples components, allowing independent evolution of the subject and its observers while maintaining synchronized behavior across the system.

Observers can be dynamically added or removed at runtime, making the system highly flexible and scalable.

10. **Behavioral Encapsulation via the State Pattern** : Explain how the State Design Pattern manages complex state transitions within an object. How does this behavioral pattern eliminate deeply nested conditional blocks (if/else or switch statements), and how does it fulfill the Single Responsibility Principle (SRP)?

Answer

The State pattern allows an object to change its behavior dynamically based on its internal state by encapsulating state-specific logic into separate classes. Instead of relying on large conditional blocks (if/else or switch statements), each state is represented by a class that defines its own behavior, and the context delegates actions to the current state object. This eliminates complex conditionals and improves readability and maintainability. It also supports the Single Responsibility Principle by ensuring that each state class is responsible for a specific behavior, resulting in a more modular and extensible system design.

The context object delegates behavior to the current state and transitions between states by changing its internal reference.

11. **Dependency Management and Development Environments :**

Explain why it is critical for a software development team to wrap up and freeze the precise version numbers of every external library package used in a project. Contrast how dependency ecosystems are declared and isolated using Python's `requirements.txt` environment workflow vs. standard Java dependency management paradigms.

Answer

It is critical for a software development team to freeze the precise version numbers of all external dependencies to ensure consistency, stability, and reproducibility across different environments. Without fixed versions, updates to external libraries can introduce unexpected behavior, compatibility issues, or system failures during development or deployment. In Python, dependency management is typically handled using a *requirements.txt* file, where each package and its exact version are explicitly listed, allowing environments to be recreated reliably using tools like *pip*. In contrast, Java commonly uses build tools such as Maven or Gradle, where dependencies and their versions are declared in configuration files (such as *pom.xml* or *build.gradle*), often with support for transitive dependencies and more structured project management. Both approaches aim to isolate environments and maintain version alignment, but Java ecosystems generally provide more formalized dependency graphs, while Python emphasizes simplicity and direct package control.

12. **Static Analysis vs. Runtime Execution** : Modern QA workflows use static analysis tools (such as Checkstyle, PMD, or SpotBugs) alongside automated testing frameworks. Define how static analysis evaluates code without executing it, and identify the types of software flaws and "code smells" it detects that runtime testing might entirely miss.

13. **White-Box vs. Black-Box Testing Methodologies** : Contrast

White-Box testing and Black-Box testing methodologies from the perspective of a software tester. Detail what structural artifacts are scrutinized under each approach, how logic paths (if/else boundaries) are verified, and why passing all black-box tests does not guarantee flawless execution.

14. **Shifting from Effective to Efficient via Profiling : Week 7**

established that code must move from being merely "Effective" to being "Efficient". Define the technical difference between these two milestones. Explain the mechanics of runtime profiling tools (like JProfiler or Py-spy) and how they isolate CPU and memory "hot spots" without altering application logic.